

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

Facultad de Ingeniería

Escuela Profesional de Ingeniería en Informática y Sistemas



PRÁCTICA N° 02:

“FUNCIONAMIENTO DE UN COMPUTADOR”

Alumno	:	Mauricio José Flores Gómez
Código	:	2018-119061
Docente	:	Dr. Hugo Manuel Barraza Vizcarra
Curso	:	Sistemas Operativos
Ciclo	:	6to Ciclo
Año de estudios	:	3er año

Tacna - Perú

2026

I. OBJETIVOS

- Analizar la arquitectura de un analizador
- Desarrollar programas en un ensamblador y analizar su ejecución de un computador 8086/88

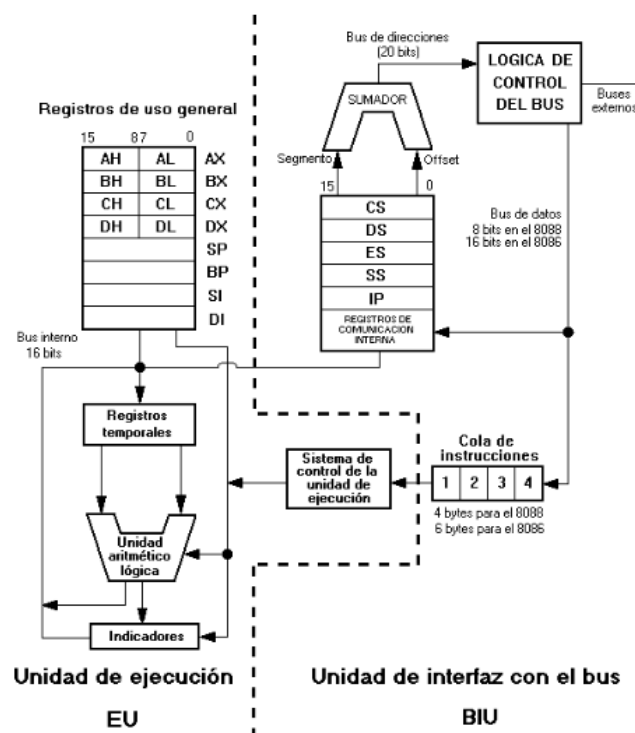
II. FUNDAMENTO TEÓRICO

El microprocesador 8086

En 1978 Intel sacó al mercado el 8086, con un bus de datos de 16 bits y capaz de direccionar hasta 1 Mb de memoria. La importancia del 8086 se debe a que fue elegido por IBM para desarrollar el Personal Computer, que fue tomado como estándar por casi todos los fabricantes de ordenadores. Posteriormente Intel fabricó sucesivamente el 80186, el 80286, el 80386, el 486 y los Pentiums, manteniendo en todos ellos la compatibilidad de software con los anteriores.

Arquitectura del procesador 8086/88

El 8086 es un microprocesador de 16 bits, tanto en lo que se refiere a su estructura como en sus conexiones externas, mientras que el 8088 es un procesador de 8 bits que internamente es casi idéntico al 8086. La única diferencia entre ambos es el tamaño del bus de datos externo. Intel trata esta igualdad interna y desigualdad externa dividiendo cada procesador 8086 y 8088 en dos sub-procesadores. O sea, cada uno consta de una unidad de ejecución (EU: Execution Unit) y una unidad interfaz del bus (BIU: Bus Interface Unit). La unidad de ejecución es la encargada de realizar todas las operaciones mientras que la unidad de interfaz del bus es la encargada de acceder a datos e instrucciones del mundo exterior. Las unidades de ejecución son idénticas en ambos microprocesadores, pero las unidades de interfaz del bus son diferentes en varias cuestiones, como se desprende del siguiente diagrama en bloques:



El ensamblador 8086/88

El lenguaje ensamblador como cualquier lenguaje de programación es un conjunto de palabras que le indican al ordenador lo que tiene que hacer. Sin embargo la diferencia fundamental es que cada instrucción escrita en lenguaje ensamblador tiene una correspondencia exacta con una operación en el procesador. Por lo que son operaciones muy sencillas tales como: "Cargar 32 en el registro BX" o "Transferir el contenido del registro CL al CH". Así pues, las palabras del lenguaje ensamblador son nemotécnicos que representan el código máquina, lenguaje que entiende el procesador.

Tamaño de los datos

En el 8086/88 se definen los siguientes tamaños de datos:

4 bits → nibble

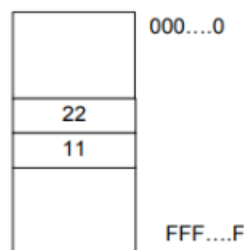
8 bits → byte

16 bits → word

32 bits → dword

Almacenamiento de datos

El 8086/88 usa el formato de almacenamiento denominado "little endian", esto quiere decir que el byte menos significativo (LSB) del dato es guardado en la parte baja de la memoria. Por ejemplo el dato 0x1122 será almacenado en memoria:



Es importante tener esto en cuenta a la hora de acceder a los datos para operar con ellos.

Segmentación

El 8086/88 tiene un ancho de bus de datos de 16 bits y un ancho de bus de direcciones de 20 bits. Con 20 bits de direcciones se puede acceder a $2^{20} = 1$ Mega posiciones de memoria. Como cada dirección de memoria contiene un byte, el total de memoria accedido por el procesador es de 1 Mbyte. El bus de datos de 16 bits lo que implica que en cada acceso a memoria se leen dos posiciones. Esta es la razón por la que es importante saber el modo de almacenamiento de los datos en memoria, visto en el apartado anterior.

El problema que se les planteó a los diseñadores fue que siendo los registros internos del procesador de 16 bits, y el bus de direcciones de 20; faltaban 4 bits para poder aprovechar al máximo las capacidades de direccionamiento del procesador. Para resolver esto, cada dirección de memoria será especificada como un segmento y un desplazamiento dentro de ese segmento. Esta solución divide la

memoria en segmentos de 64 K, lo cual limitó bastante los diseños de los procesadores posteriores de la familia (80286, 80386 etc.); aunque posteriormente se idearon métodos para resolver este problema, como la memoria extendida (no compatible, por supuesto, con el 8086/88).

Registros internos del 8086/88

Los registros de datos son de 16 bits, aunque están divididos. lo que permite su acceso en 8 bits. Estos registros son de propósito general aunque todos tienen alguna función por defecto.

AX (acumulador) se usa para almacenar el resultado de las operaciones, es el único registro con el que se puede hacer divisiones y multiplicaciones. Puede ser accedido en 8 bits como AH para la parte alta (HIGH) y AL (LOW) para la parte baja.



BX (registro base) almacena la dirección base para los accesos a memoria. También puede accederse como BH y BL, parte alta y baja respectivamente.

CX (contador) actúa como contador en los bucles de repetición. CL (parte baja del registro) almacena el desplazamiento en las operaciones de desplazamiento y rotación de múltiples bits.

DX (datos) es usado para almacenar los datos de las operaciones.

IP (índice de programa) almacena el desplazamiento dentro del segmento de código. Este registro junto al registro CS apunta a la dirección de la próxima instrucción. No puede ser usado como operando en operaciones aritmético/lógicas.

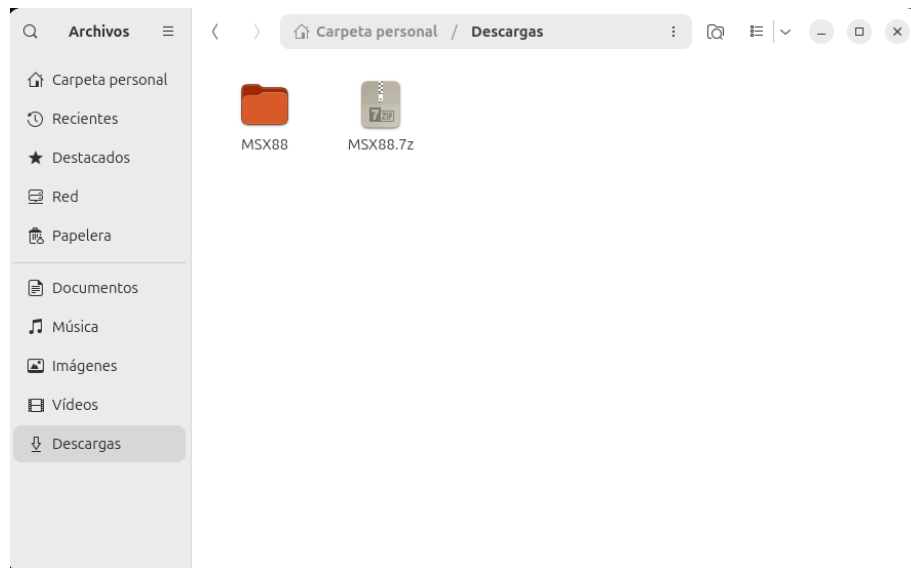
SP (índice de pila) almacena el desplazamiento dentro del segmento de pila, y apunta al último elemento introducido en la pila.

III. PROCEDIMIENTO

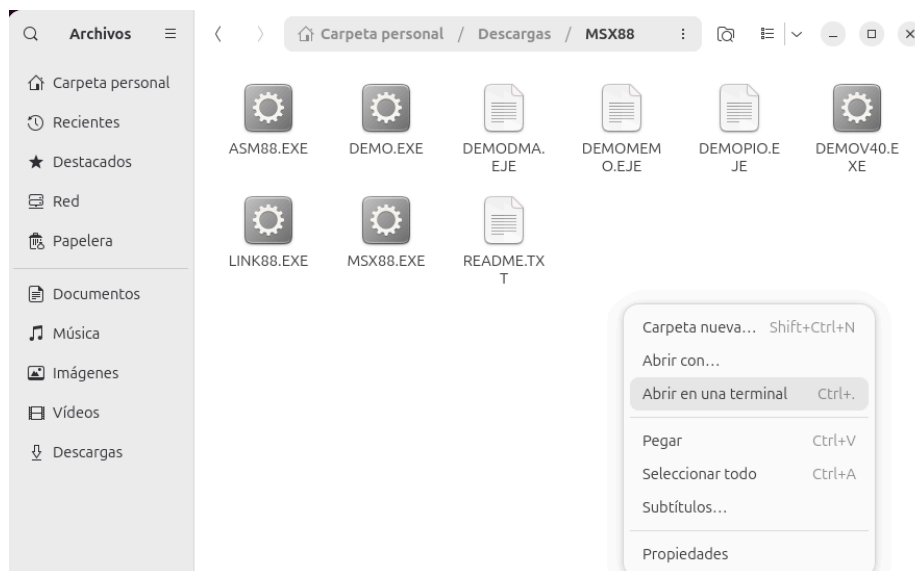
Abrimos la máquina virtual de Ubuntu y verificamos que tenga instalado el DOSBox.

La imagen es una captura de pantalla de una terminal de Ubuntu. El prompt de usuario es 'unjbg@unjbg-VirtualBox:~/Descargas/MSX88\$'. El comando ejecutado es 'dosbox -version'. La salida muestra: 'DOSBox version 0.74-3, copyright 2002-2019 DOSBox Team.' seguido de un mensaje de copyright: 'DOSBox is written by the DOSBox Team (See AUTHORS file))', 'DOSBox comes with ABSOLUTELY NO WARRANTY. This is free software, and you are welcome to redistribute it under certain conditions; please read the COPYING file thoroughly before doing so.'

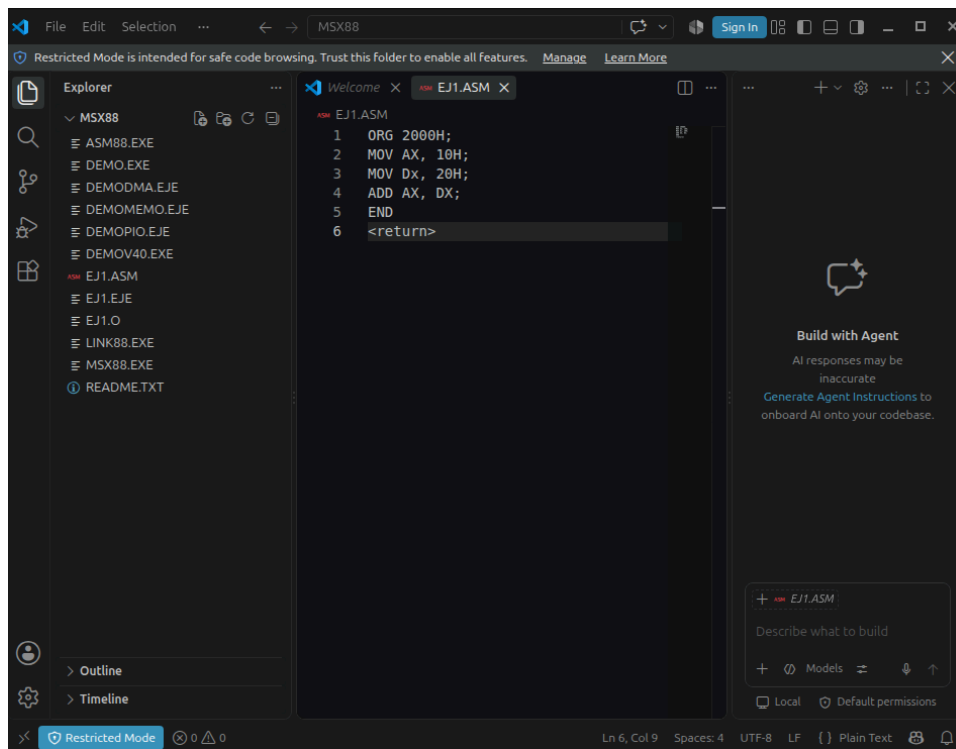
Descargamos el comprimido MSX88.7z del Aula virtual y descomprimos en la carpeta de Descargas.



Accedemos a la carpeta MSX88 e invocamos a la Terminal desde el menú contextual.

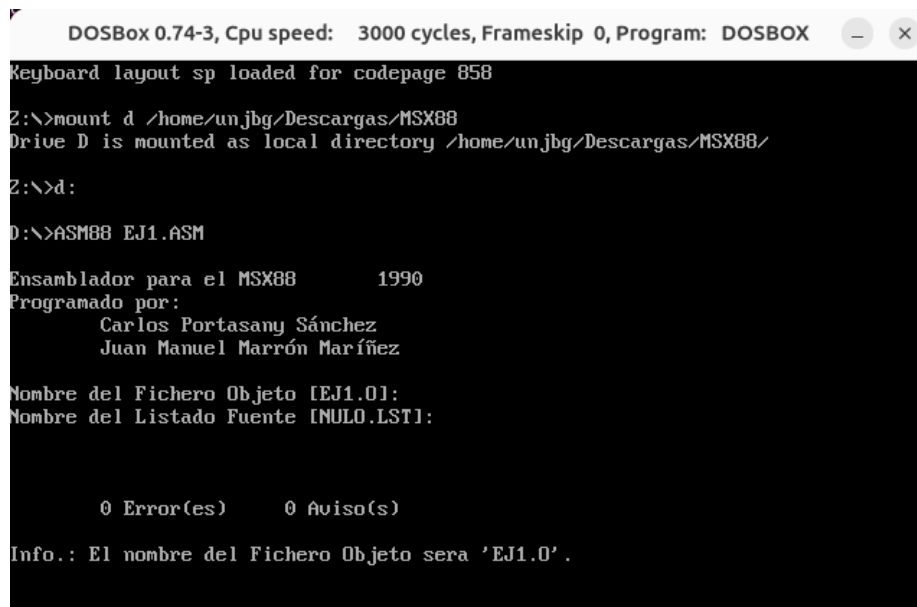


En el terminal, abrimos un editor de texto y crearemos un fichero llamado EJ1.ASM con el siguiente código ensamblador



Luego ejecutaremos el Emulador DOSBox y al escribir los comandos:

Ensamblamos el fichero anterior



Ahora enlazamos el fichero EJ1.O y visualizamos el contenido de los nuevos archivos creados: el archivo objetivo EJ1.O y el ejecutable EJ1.EJE

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Info.: El nombre del Fichero Objeto sera 'EJ1.0'.

D:\>LINK88 EJ1.0

Montador para el MSX88 1990
Programado por:
    Juan Manuel Marrón Marfínez
    Carlos Portasany Sánchez

Nombre del Fichero Ejecutable [EJ1.EJE]:

Info.: El nombre del fichero ejecutable sera 'EJ1.EJE'.

D:\>type EJ1.0
2000F308B81000BA200003C2

D:\>type EJ1.EJE
Fichero Ejecutable!!

200008F3B81000BA200003C2

```

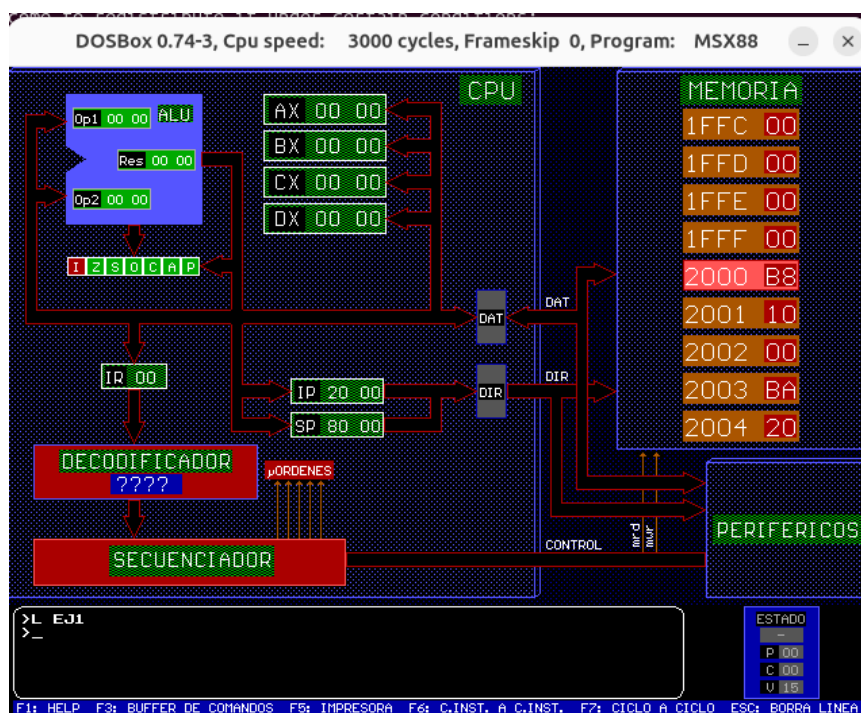
¿Son iguales EJ1.OBJ y EJ1.EXE?

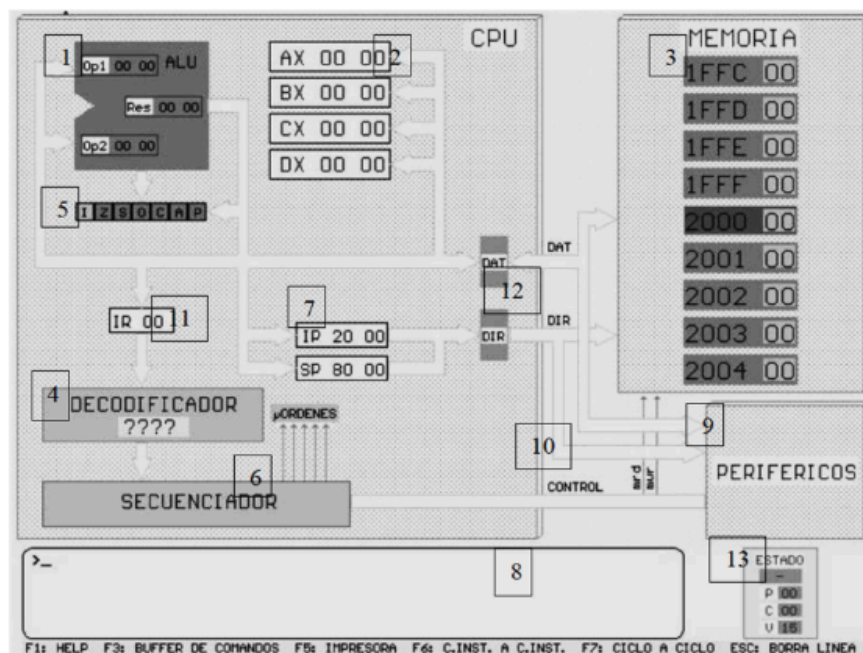
No, no son iguales. El archivo EJ1.OBJ contiene el código objeto generado por el ensamblador, mientras que EJ1.EXE es el archivo ejecutable generado después del proceso de enlace.

Identificamos las instrucciones en código máquina. **¿En qué sistema numérico están representadas las instrucciones en código máquina?**

Están representadas en sistema hexadecimal.

Ejecutamos el MSX88 y cargamos el programa EJ1.EJE en el simulador.y escribimos >L EJ1





Responde las siguientes preguntas:

1. Dada la pantalla del simulador MSX88. Describa brevemente cada uno de sus componentes (según la numeración planteada)

1. **ALU:** Realiza las operaciones aritméticas y lógicas del procesador.
2. **Registros:** Almacenan temporalmente datos e información necesaria para la ejecución de las instrucciones.
3. **Memoria:** Almacena las instrucciones y los datos que utiliza el programa.
4. **Decodificador:** Interpreta la instrucción contenida en el registro de instrucciones.
5. **Registro de estado (flags):** Almacena indicadores que muestran el resultado de las operaciones realizadas por la ALU.
6. **Secuenciador:** Controla y coordina la ejecución de las instrucciones mediante señales de control.
7. **IP y SP:** el IP indica la dirección de la siguiente instrucción a ejecutar y el SP indica la posición actual de la pila.
8. **Pantalla de órdenes:** Permite introducir comandos y visualizar la información del simulador.
9. **Periféricos:** Representan los dispositivos externos con los que puede interactuar el procesador.
10. **Bus de control:** Transporta las señales que permiten controlar las diferentes operaciones del sistema.
11. **IR (Registro de Instrucciones):** Almacena la instrucción que está siendo ejecutada.
12. **Registros DAT y DIR:** Permiten manejar los datos y las direcciones utilizadas para acceder a la memoria.
13. **Estado:** Muestra información sobre el estado actual de la ejecución del procesador.

2. ¿Qué tamaño tienen los registros?

Los registros principales del simulador MSX88 tienen un tamaño de 16 bits.

3. ¿Qué tamaño (en bits) tiene el bus de direcciones?

El bus de direcciones tiene un tamaño de 16 bits.

4. ¿Cuál es el tamaño de la memoria?

La memoria direccionable es de 64 KB, equivalente a 524 288 bits.

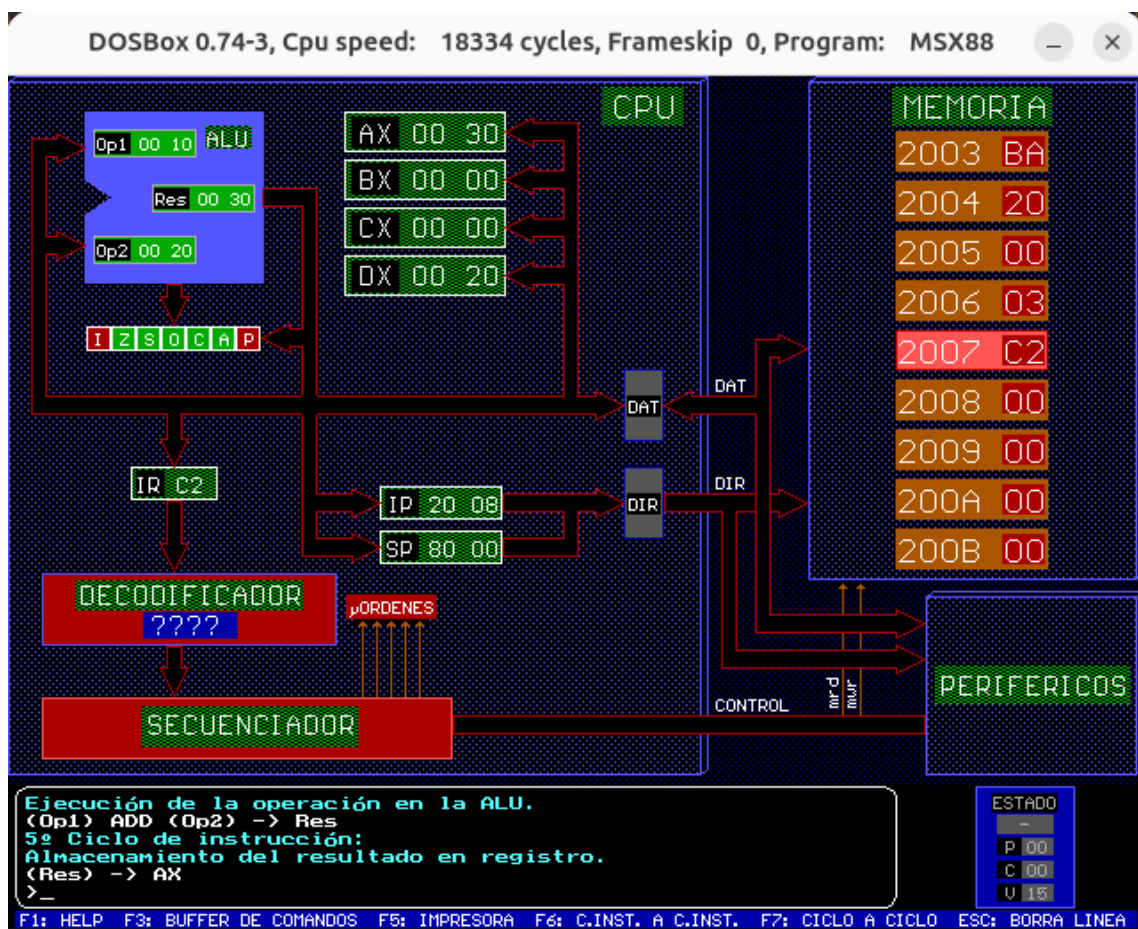
5. Observe el registro contador de programa o Puntero de instrucción IP ¿Cuál es su valor?

El registro IP tiene el valor 2000H.

6. ¿Qué significa IP?

IP significa Instruction Pointer (Puntero de Instrucción) y señala la dirección de memoria de la siguiente instrucción que será ejecutada.

Comenzamos la simulación, pulsamos la tecla F6 para ejecutar instrucción a instrucción o usando la tecla F7 para ejecutar cada instrucción en cada uno de sus ciclos de máquina (por fases). Analizamos cada uno de los pasos para la ejecución de las instrucciones escritas al inicio de la práctica.



Luego de haber finalizado la ejecución del programa respondemos las siguientes preguntas:

- **El Registro de Instrucción es el IR. ¿Para qué sirve?**

El registro de instrucción (IR) sirve para almacenar temporalmente la instrucción que está siendo leída de la memoria y que será interpretada y ejecutada por el procesador.

- **Según se van ejecutando las instrucciones, ¿qué sucede con el registro Puntero de Instrucción IP?**

El registro IP se va modificando conforme se ejecutan las instrucciones, avanzando hacia la siguiente posición de memoria donde se encuentra la próxima instrucción del programa.

- **Una vez finalizada la última instrucción del programa, ¿cómo se han modificado los registros AX y DX? ¿Cuál es el resultado de la operación? ¿Dónde se ha almacenado?**

Al finalizar el programa, AX tiene un registro de 30H y DX tiene un registro de 20H. El resultado de la operación es: $10H + 20H = 30H$. El resultado 30H queda almacenado en AX.

IV. ANÁLISIS DE RESULTADOS

1. Cambia los datos que se suman por los datos 1234h y 1000h. Repetir los pasos del procedimiento hasta terminar la simulación.

Al cambiar los datos a 1234h y 1000h, se realizó la suma de ambos valores. El resultado obtenido fue 2234h, almacenándose en el registro AX, mientras que el registro DX conservó el valor 1000h.

2. Cambia la operación aritmética. En vez de sumar se desea restar. Para ello, cambiar la instrucción ADD AX, DX por SUB AX, DX en el código. Repetir los pasos hasta terminar la simulación y contestar las siguientes preguntas:

- Una vez finalizada la última instrucción del programa: **¿Cómo se han modificado los registros AX y DX? ¿Cuál es el resultado de la operación? ¿Dónde se ha almacenado?**

Una vez finalizada la ejecución, el registro AX contiene el valor 0234h, mientras que el registro DX conserva el valor 1000h. El resultado de la operación SUB AX, DX es 0234h, y este queda almacenado en el registro AX.

- Por simplificar, los datos que vemos están en hexadecimal en vez de en binario. **Pase el resultado final de la operación a binario. ¿En qué formato está representado?**

El resultado final de la operación es 0234h. Al convertir de hexadecimal a binario se obtiene 0000 0010 0011 0100. El resultado está representado en formato binario de 16 bits.

3. Cambie el orden de los operandos en el programa anterior. Escribir SUB DX, AX y observar qué pasa con los flags de estado del computador.

Al cambiar el orden de los operandos y utilizar la instrucción SUB DX, AX, se realiza la operación $1000h - 1234h$, obteniéndose un resultado negativo. El resultado almacenado en el registro DX es EDCCh.

- Revise los registros de flags. Investigue: **¿Qué valor tiene el bit de signo S? ¿Y el de paridad P? ¿Por qué?**

El bit de signo $S = 1$, porque el resultado es negativo. El bit de paridad $P = 1$, porque el byte menos significativo del resultado CCh contiene cuatro bits en 1, es decir, una cantidad par de unos.

V. CONCLUSIONES

- Se logró analizar la arquitectura del computador 8086/88, comprendiendo el funcionamiento de sus principales registros y el proceso que sigue el procesador durante la ejecución de las instrucciones.
- Se desarrollaron y ejecutaron programas en lenguaje ensamblador, permitiendo observar mediante el simulador los cambios en los registros y comprobar el funcionamiento de las operaciones realizadas por el computador 8086/88.